

Complexity-Guide d *Pre-Pretraining*

*Can We Make Language Models Better by
Teaching Them Baby Puzzles First?*

Nicholas Kluge Corrêa
Postdoc @ Bonn-Aachen International Center for Information Technology (b-it)

What if, before you ever taught a language model to read English, you first taught it to recognize simple patterns — like ABABAB or ABCCBA — Would that make it better at English later?

Our Roadmap for the next ~30 min ...

Why would you do this?

>> I'll (try to) convince you that “pre-pretraining” is an idea worth thinking about.

What does complexity even mean?

>> Present some tools that can be used to categorize data.

How do we run the experiments?

>> I'll show you what are the kind of experiments we are running?

What did we find?

>> Next presentation? Or skip straight to the current results ([SPOILERS](#))

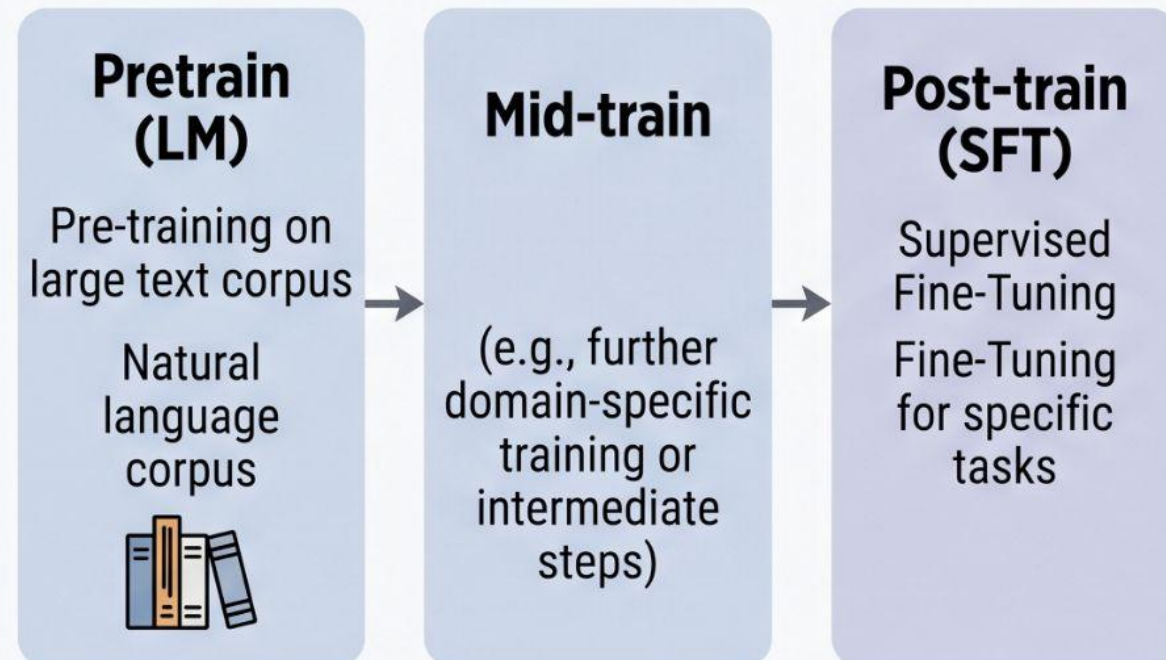
Why would you do this?

Because babies don't start with Tolstoy. They start with peek-a-boo! And there is some supporting evidence to support the idea we should approach LLMs in a similar fashion ...

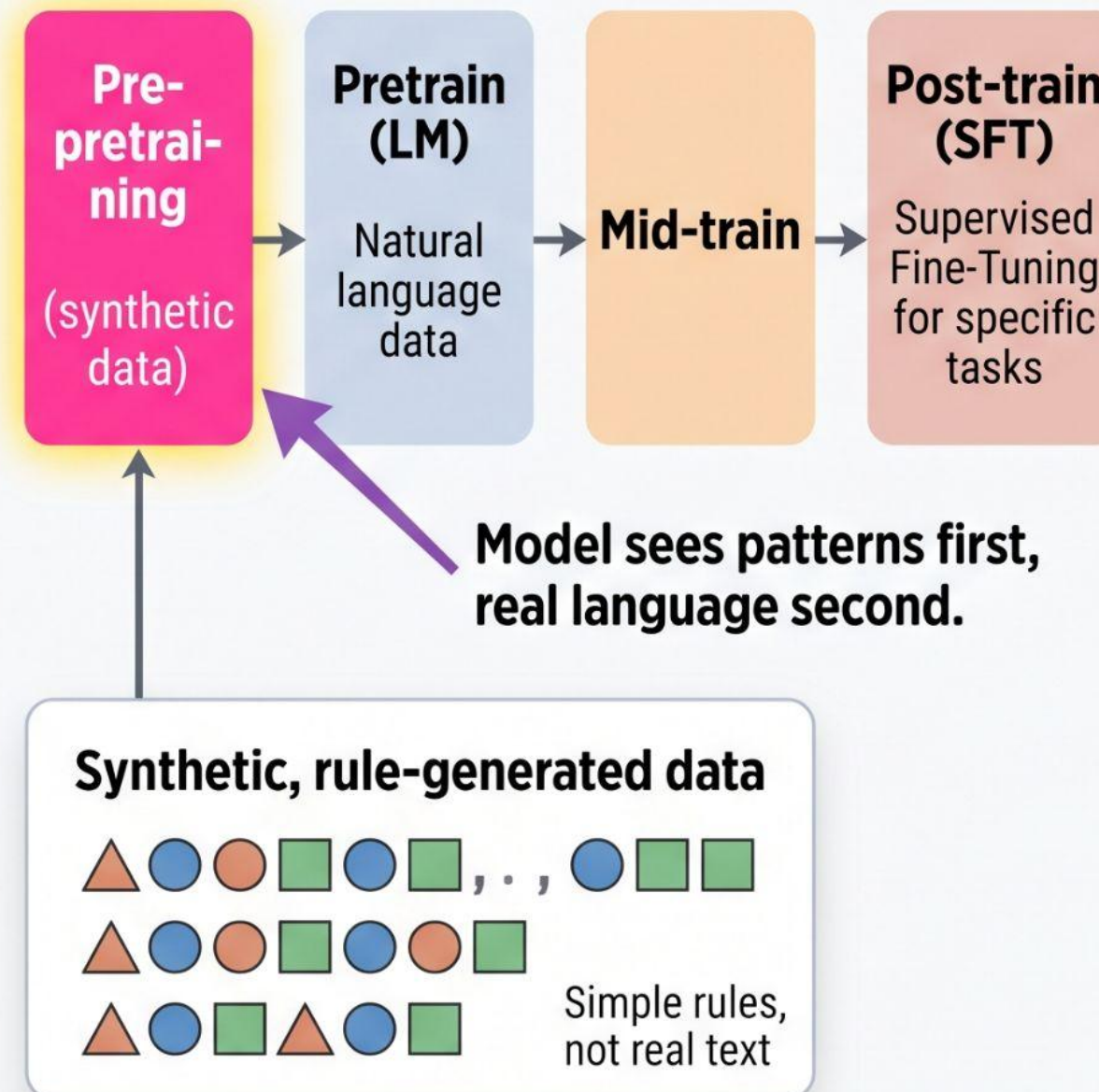
- >> Program synthesis improves with synthetic pre-training ... ([1509.00413](#))
- >> Math reasoning benefits from formal language exposure ... ([1904.01557](#))
- >> Models trained without ANY human language still develop linguistic capabilities ... ([2012.11995](#))
- >> Pre-training on formal languages imparts measurable linguistic biases ... ([2304.13060](#))

Learning simple structure first makes models better at learning complex structure later ...

TRADITIONAL PIPELINE



WITH PRE-PRETRAINING



**Can we make this
systematic? Can we
measure the structure in
the data, and use those
measurements to predict
what will transfer best?**

What does complexity even mean?

Algorithmic Information Theory (AIT) concerns itself with the relationship between computation and information.

A: "01..."

What does complexity even mean?

Algorithmic Information Theory (AIT) concerns itself with the relationship between computation and information.

```
A:  "010101010101010101010101010101010101010101..." ← print("01" * 32)
```


What does complexity even mean?

Algorithmic Information Theory ([AIT](#)) concerns itself with the relationship between computation and information.

A: "01010101010101010101010101010101..." ← `print("01" * 32)`

B: "11001000011000011101111011101100..."

What does complexity even mean?

Algorithmic Information Theory ([AIT](#)) concerns itself with the relationship between computation and information.

A: `"010101010101010101010101010101010101010101..."` ← `print("01" * 32)`

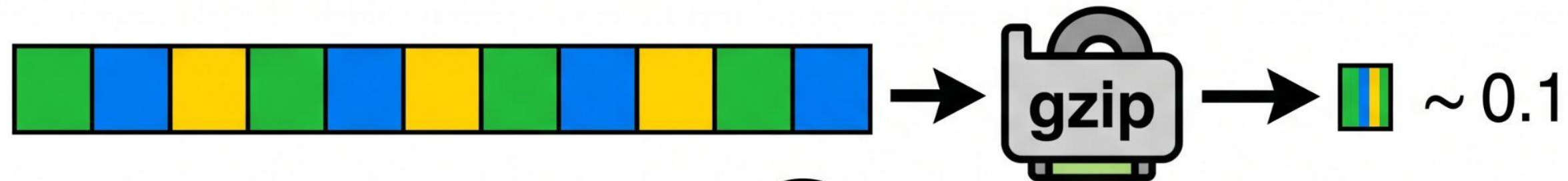
B: `"11001000011000011101111011101100..."` ← `print("11001000...")`

Sadly, Kolmogorov complexity is incomputable — you can't write a program that finds the shortest program for any input. So we need an approximation.

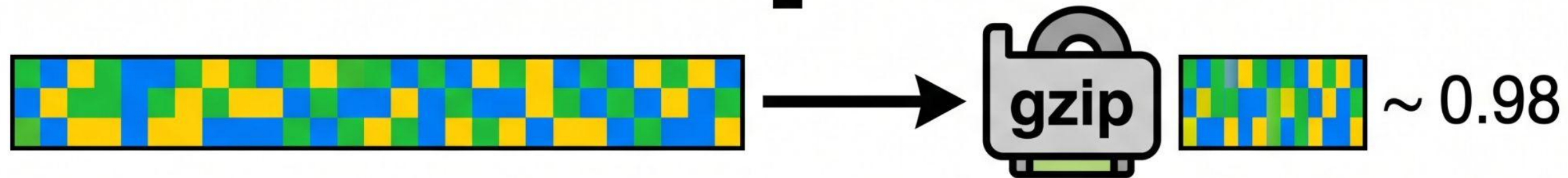
Kolmogorov complexity $K(x)$: “Shortest program that outputs x ”

↓ INCOMPUTABLE

gzip compression ratio: $\text{size}(\text{compressed}) / \text{size}(\text{original})$

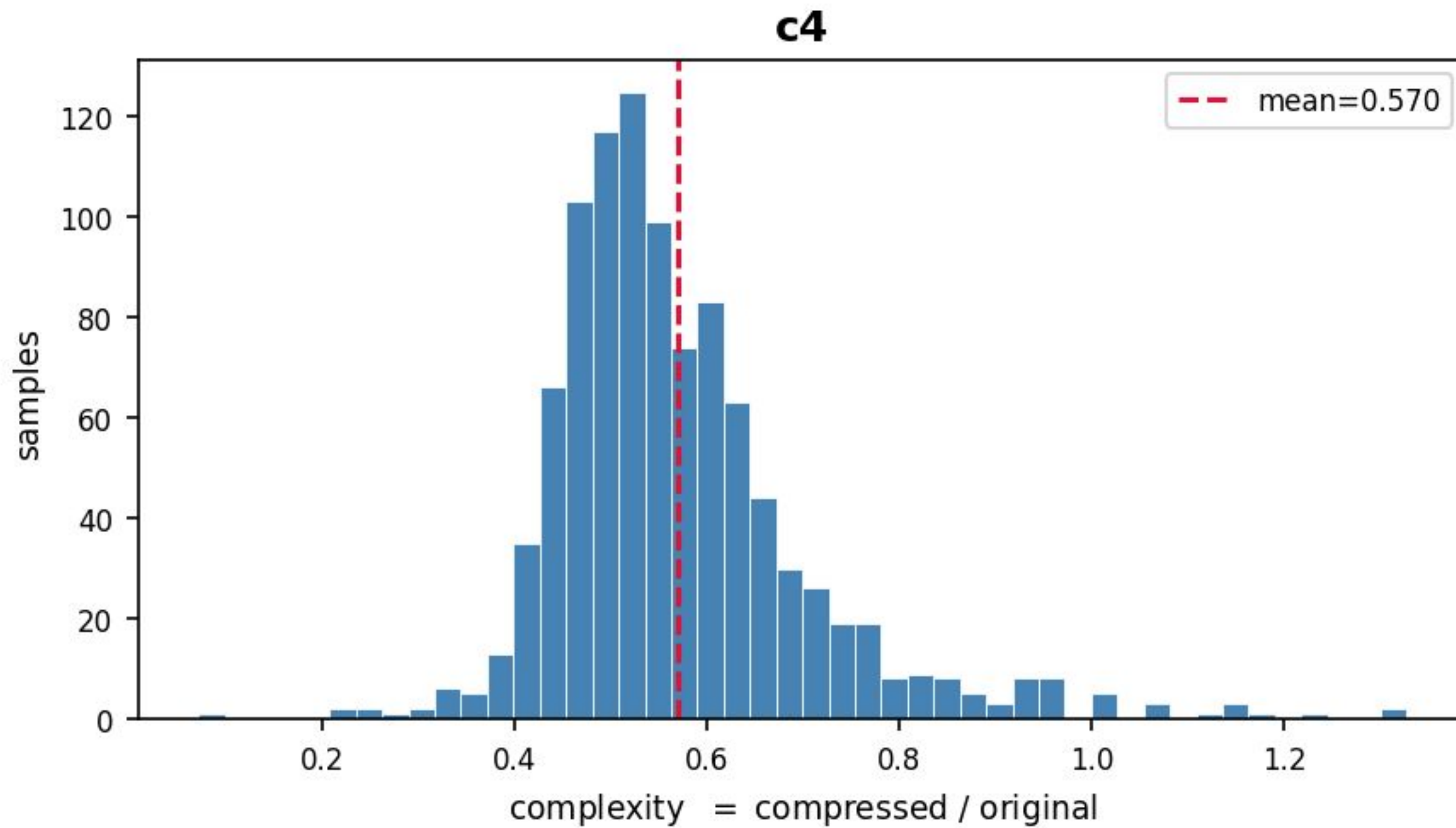


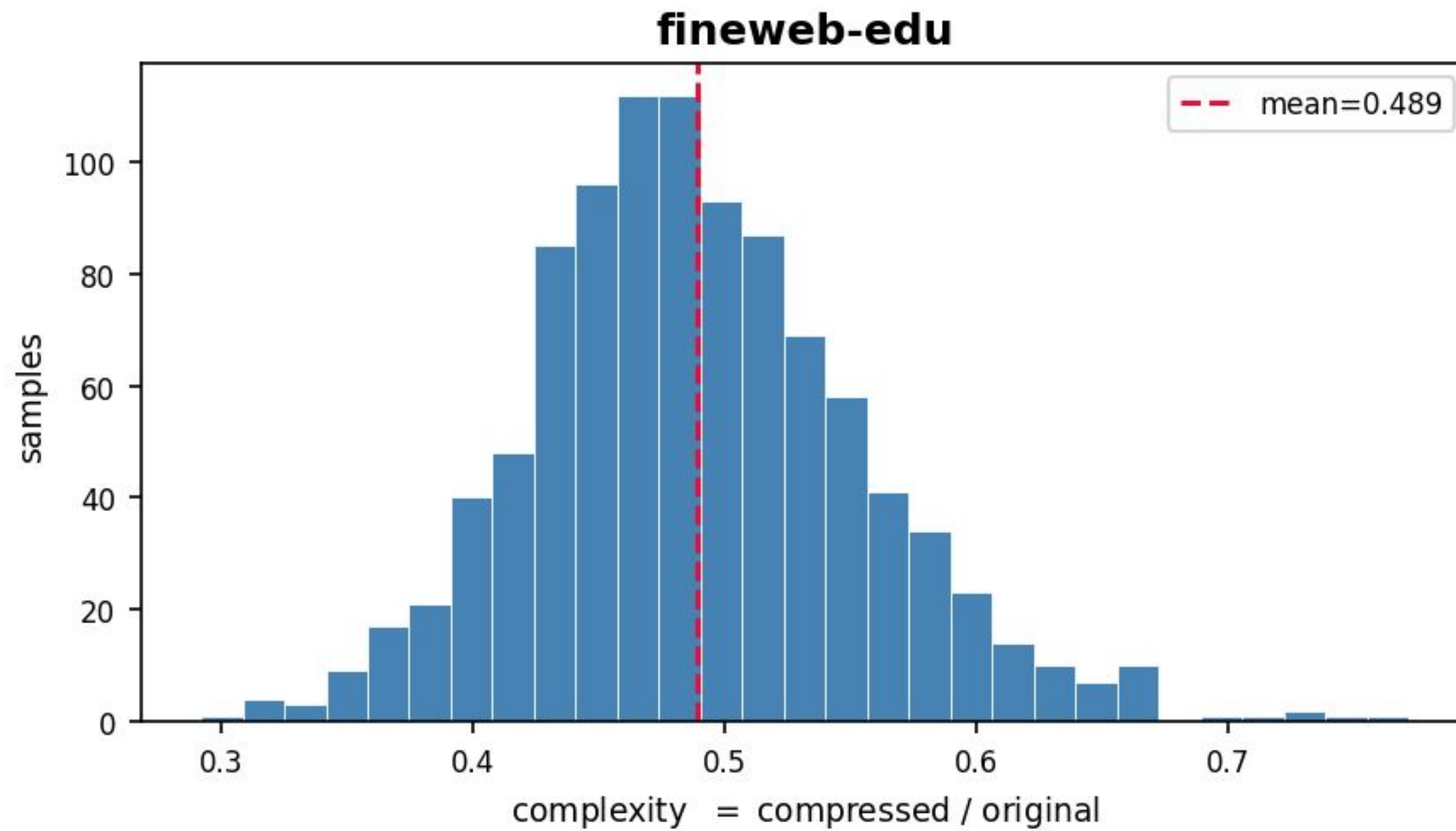
$K(x)$? We approximate the uncomputable with something practical.



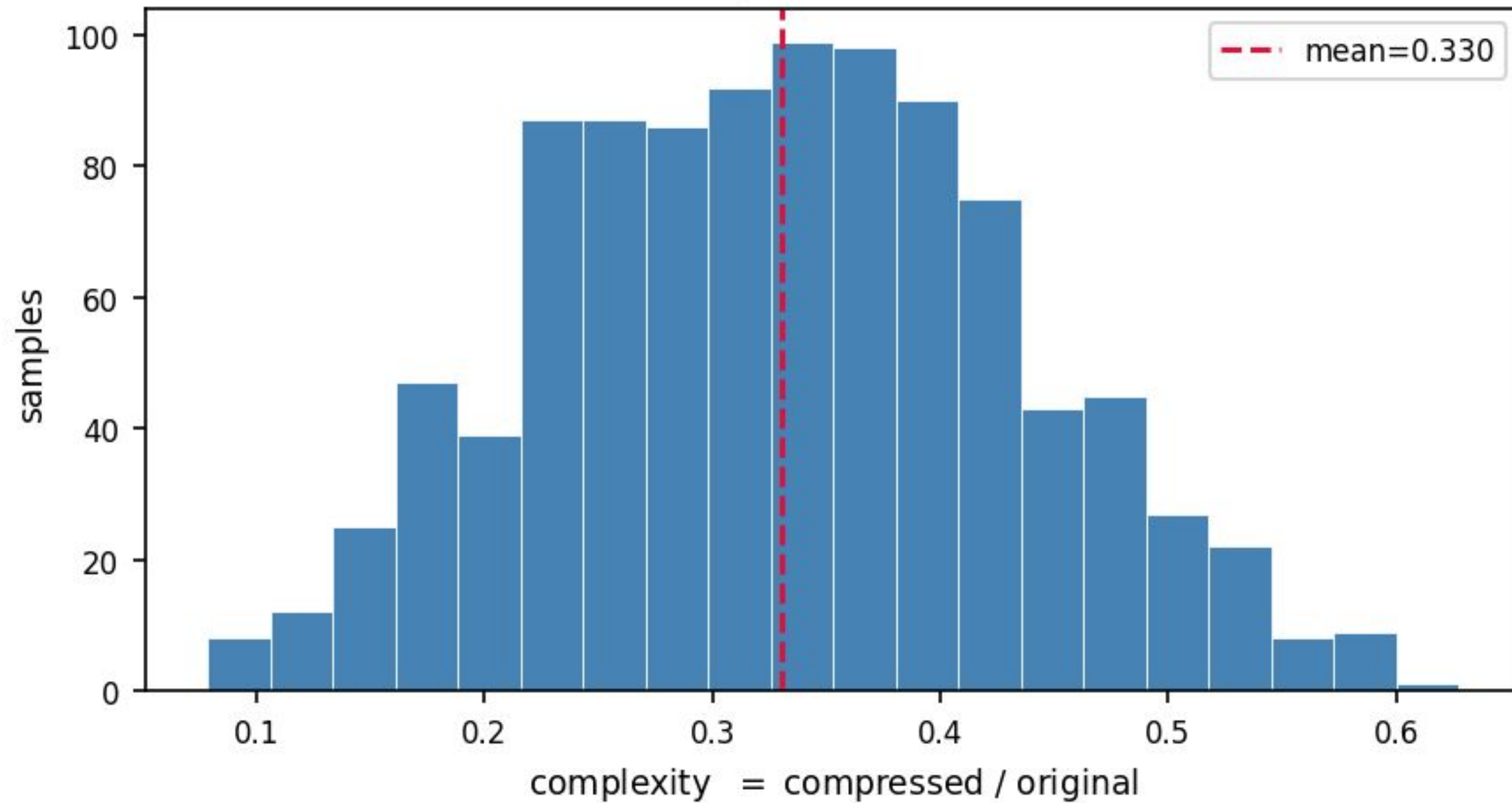
gzip ratio $\rightarrow 0.0$ “Very compressible”

gzip ratio $\rightarrow 1.0$ “Incompressible”

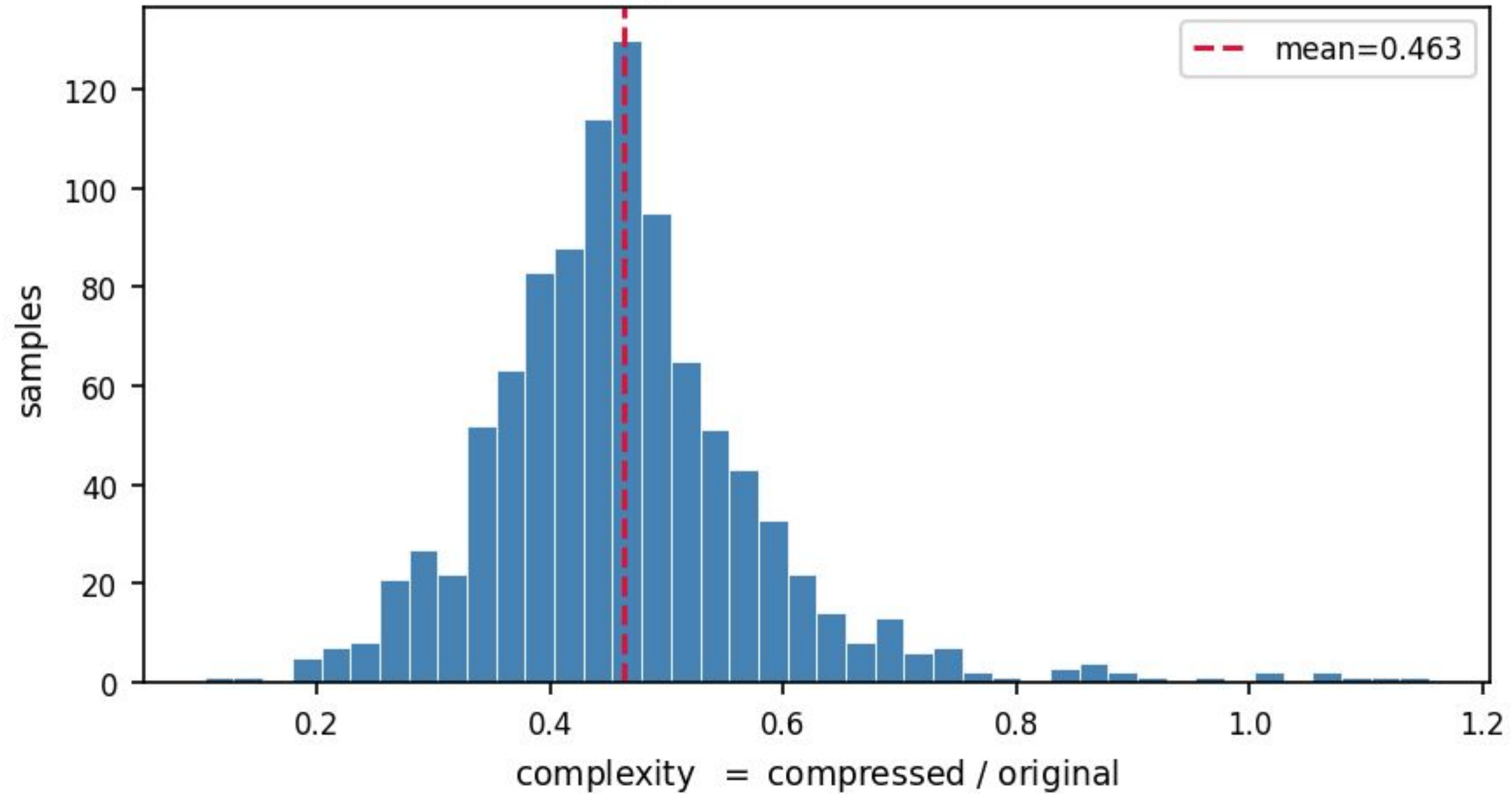


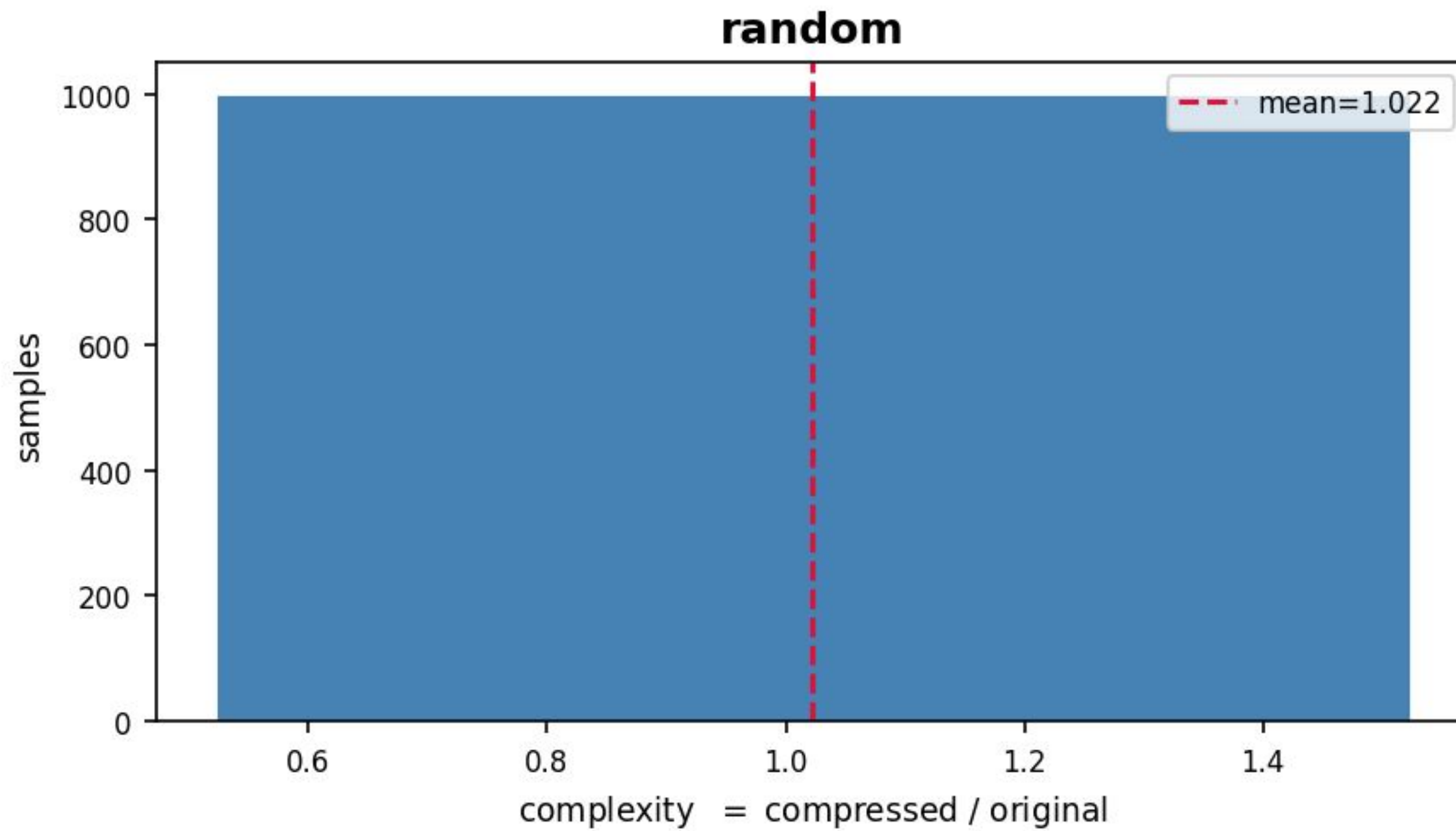


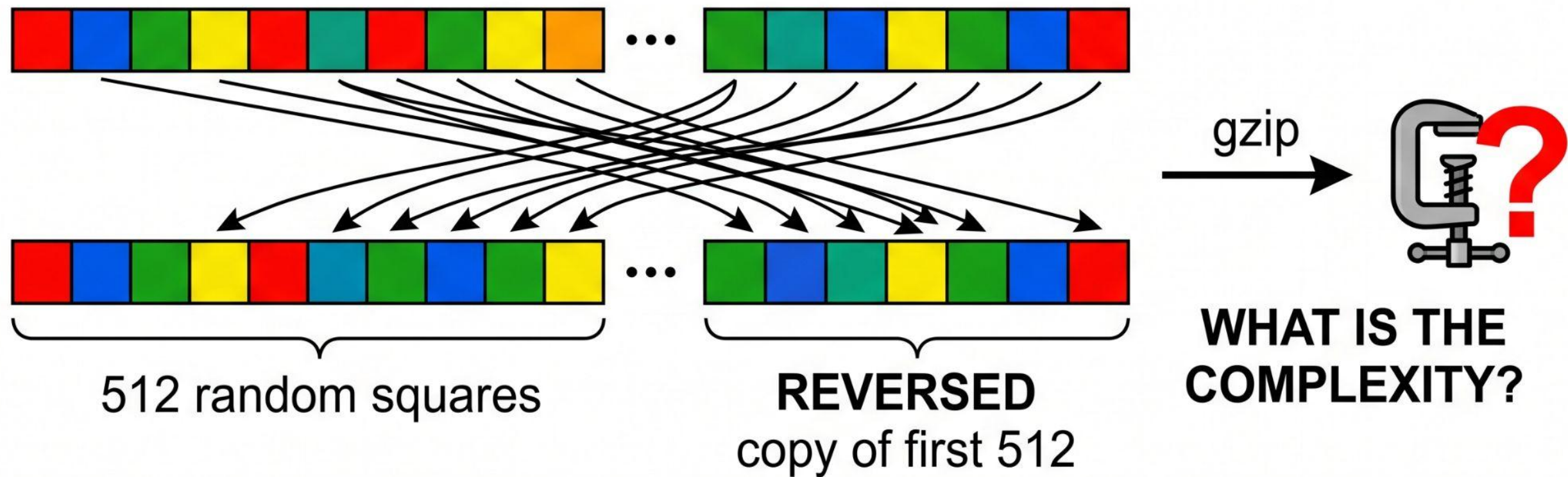
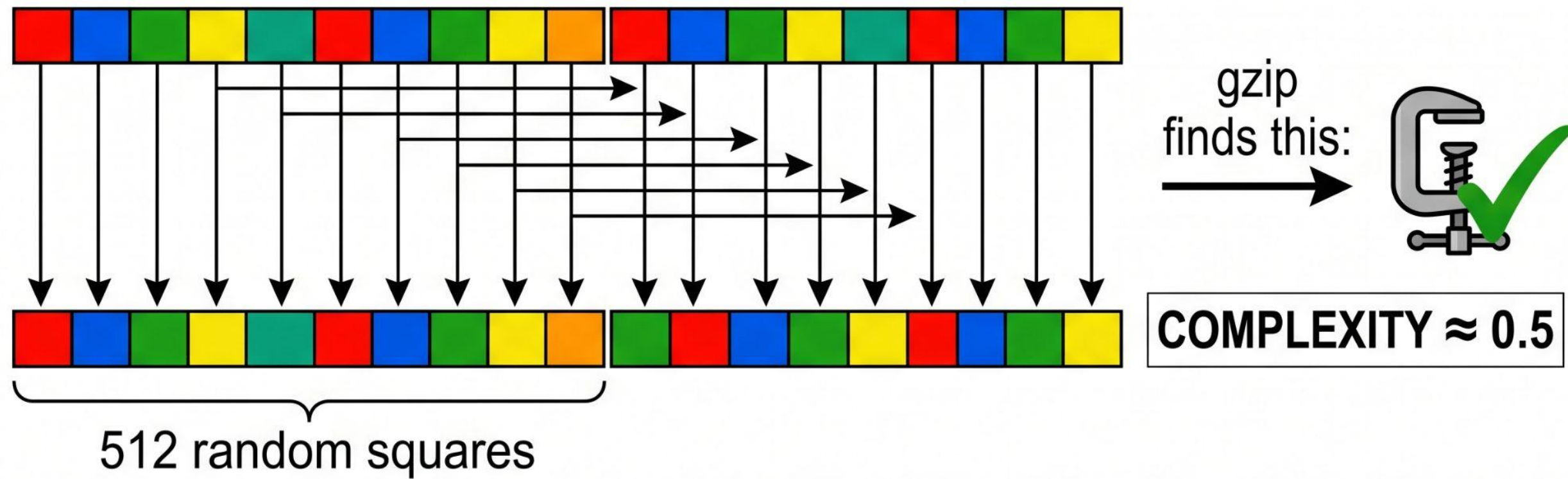
codeparrot-clean



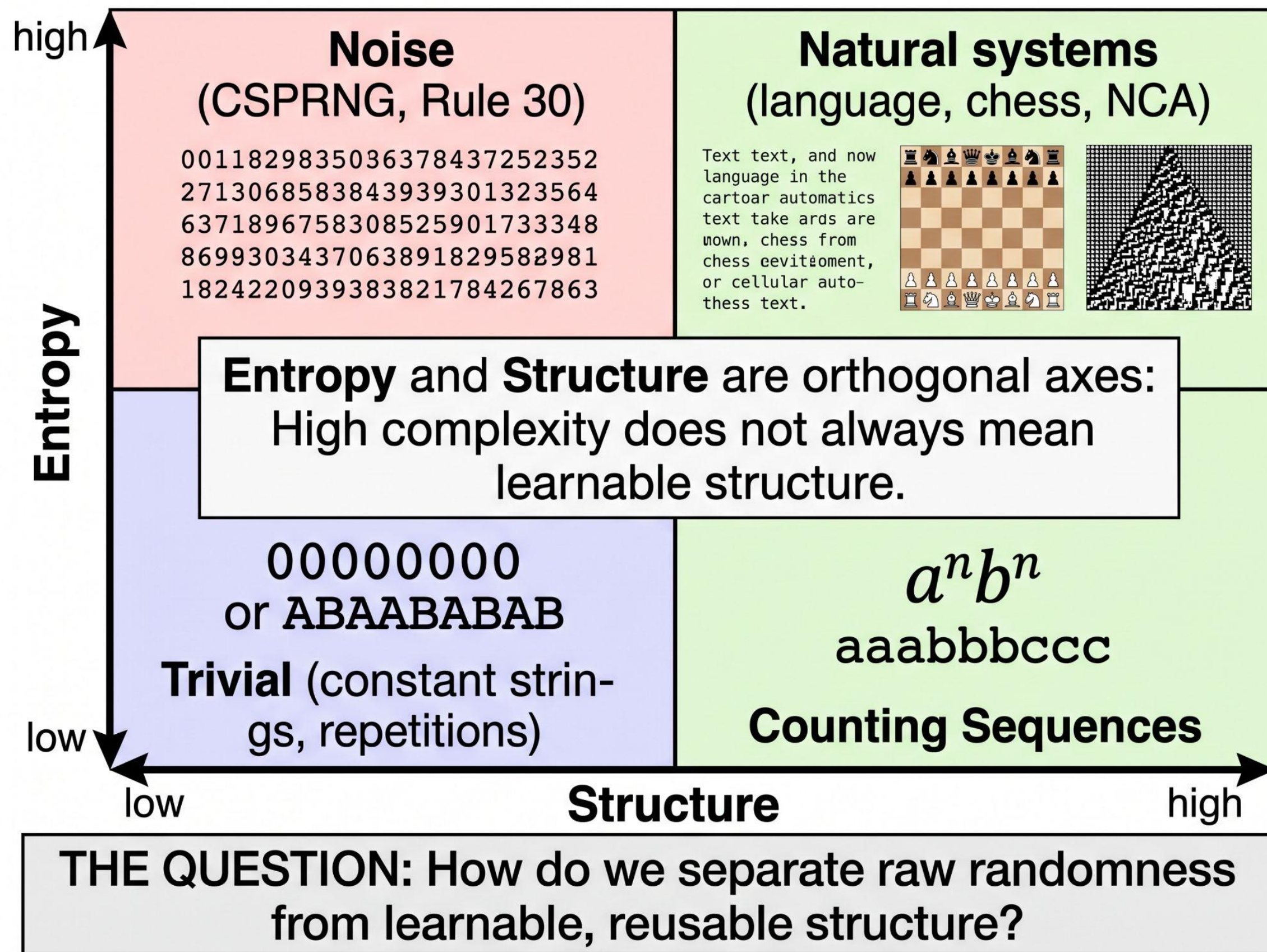
open-web-math







**Complexity is
observer-relative. But
complexity is not the
same as learnable
structure.**

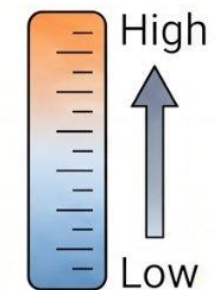


Epiplexity ([2601.03220](#))
measures the structural,
learnable information
that a computationally
bounded observer can
extract from data.

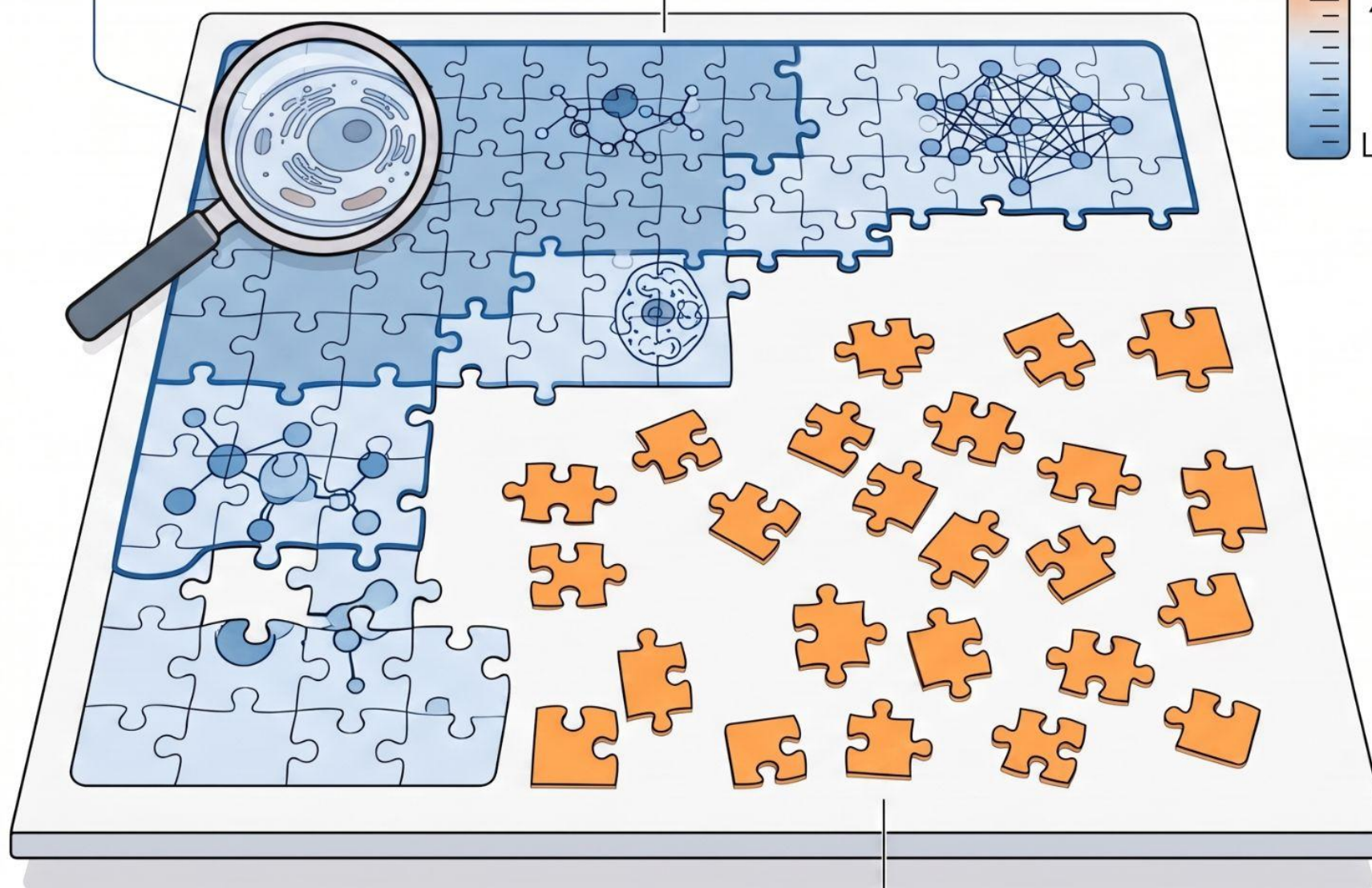
S_t : Learned Structure
(Coherent and Predictable Information)

Structure already assembled represents what the observer has successfully learned.

Compute Budget

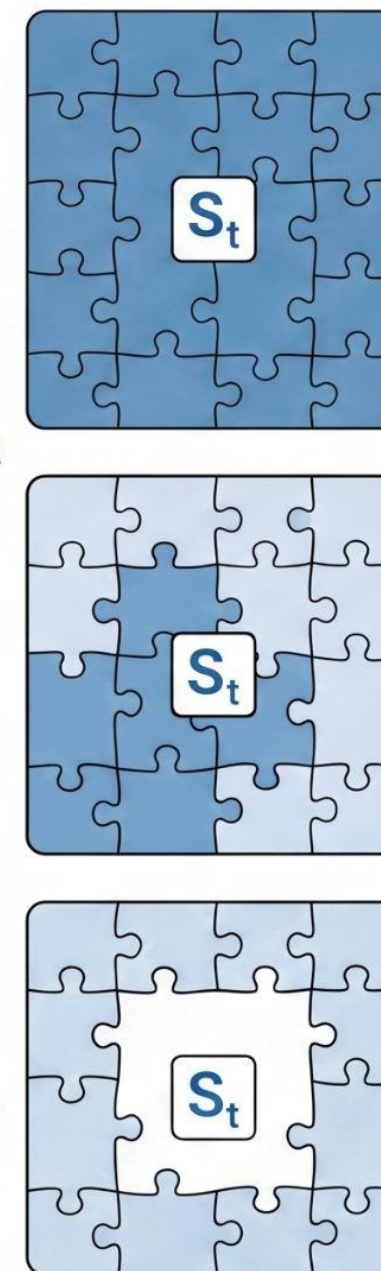


Increasing Computation

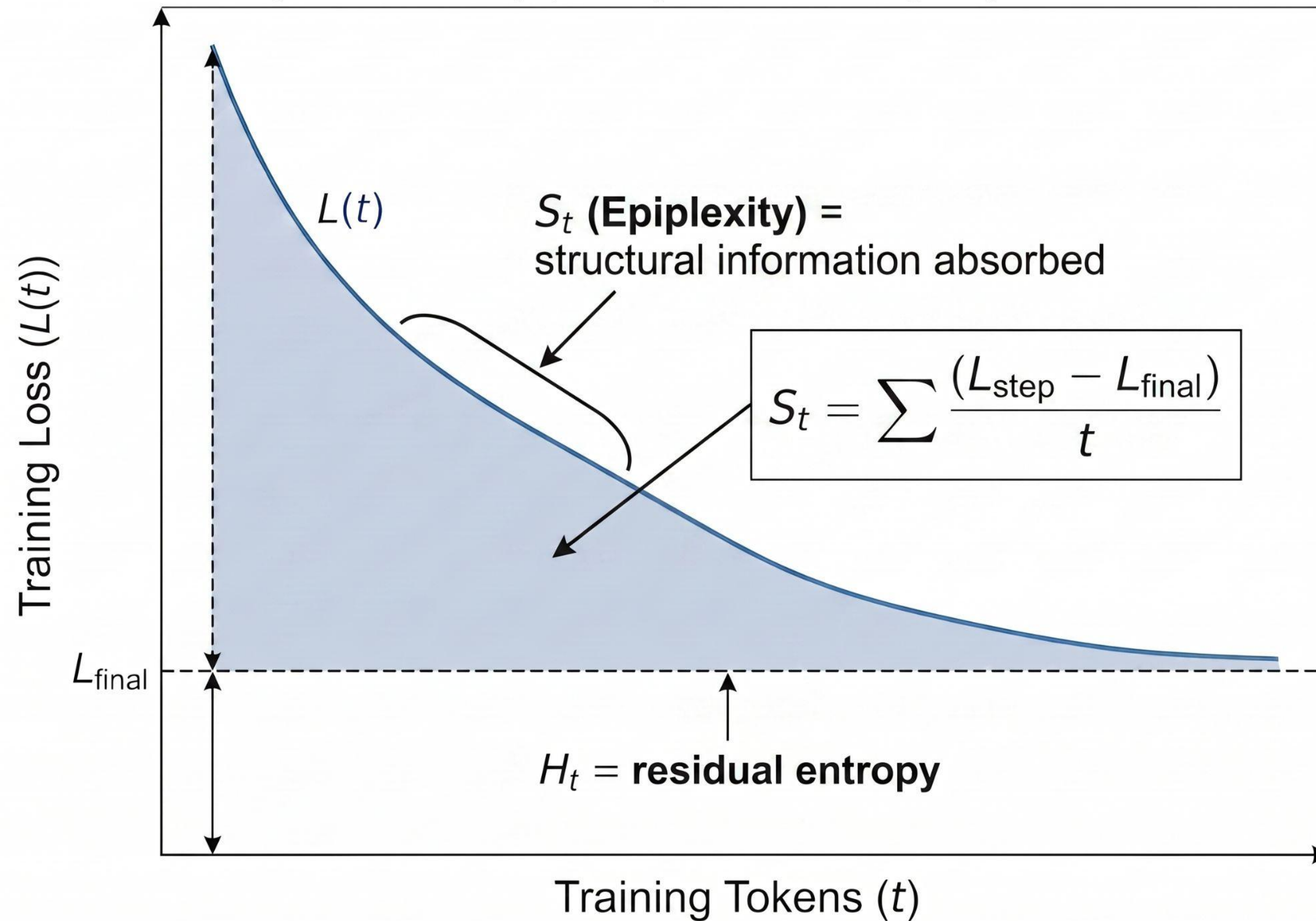


■ **H_t : Unpredictable Information**
(Scattered Data Pieces)

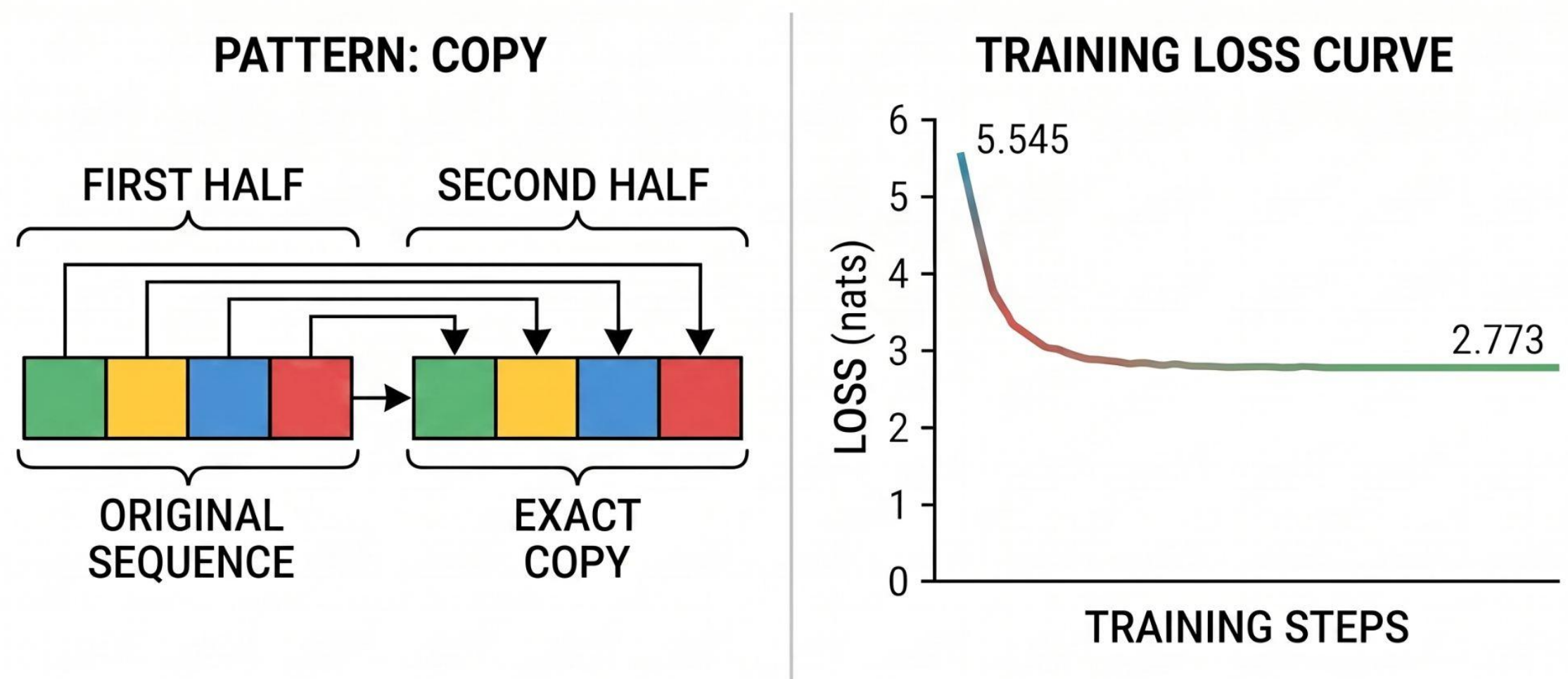
Remaining pieces are part of the same structure, just not yet connected under the available budget.



You already measure epiplexity — it's hiding in your loss curves.



For a pattern where the second half is an exact copy of the first half, epiplexity tells a clear story: half the information is learnable structure.



QUANTITATIVE ANALYSIS

Vocabulary: 256 symbols

Uniform loss: $\ln(1/256) = -5.545$ nats

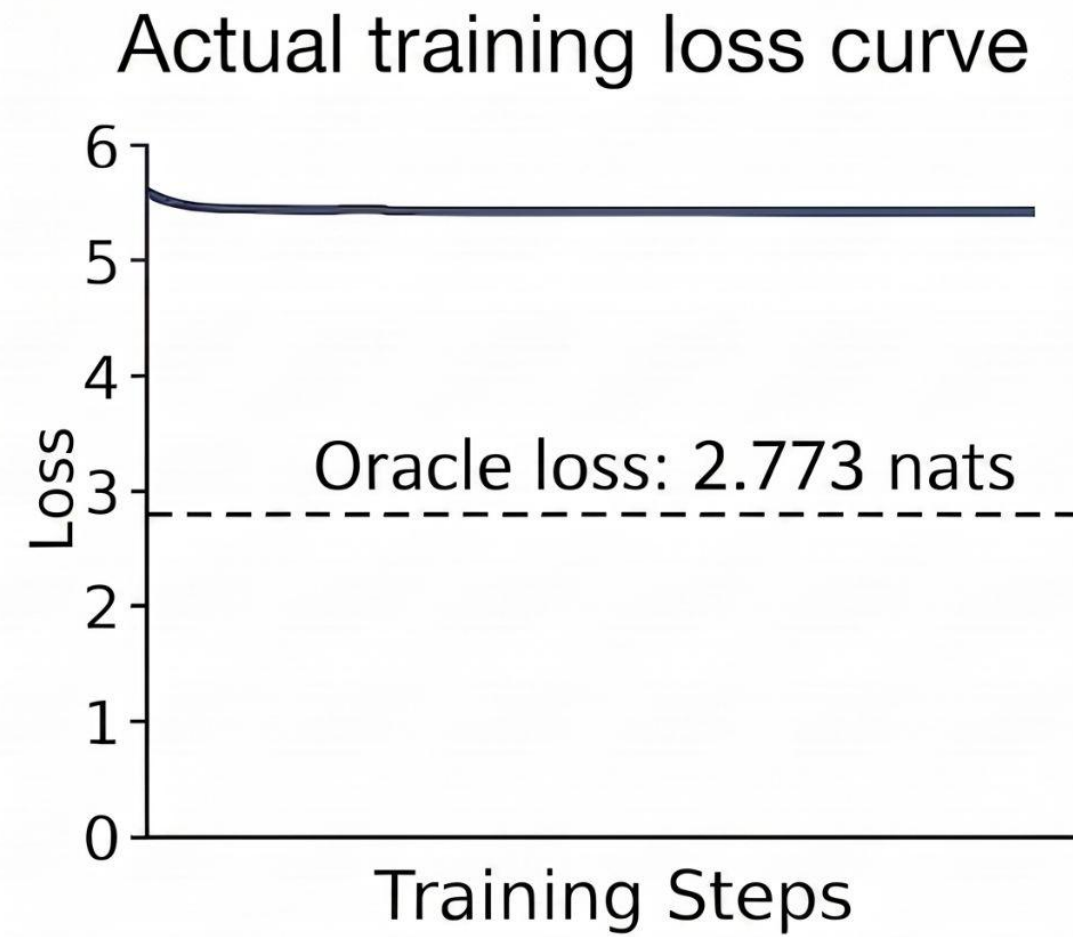
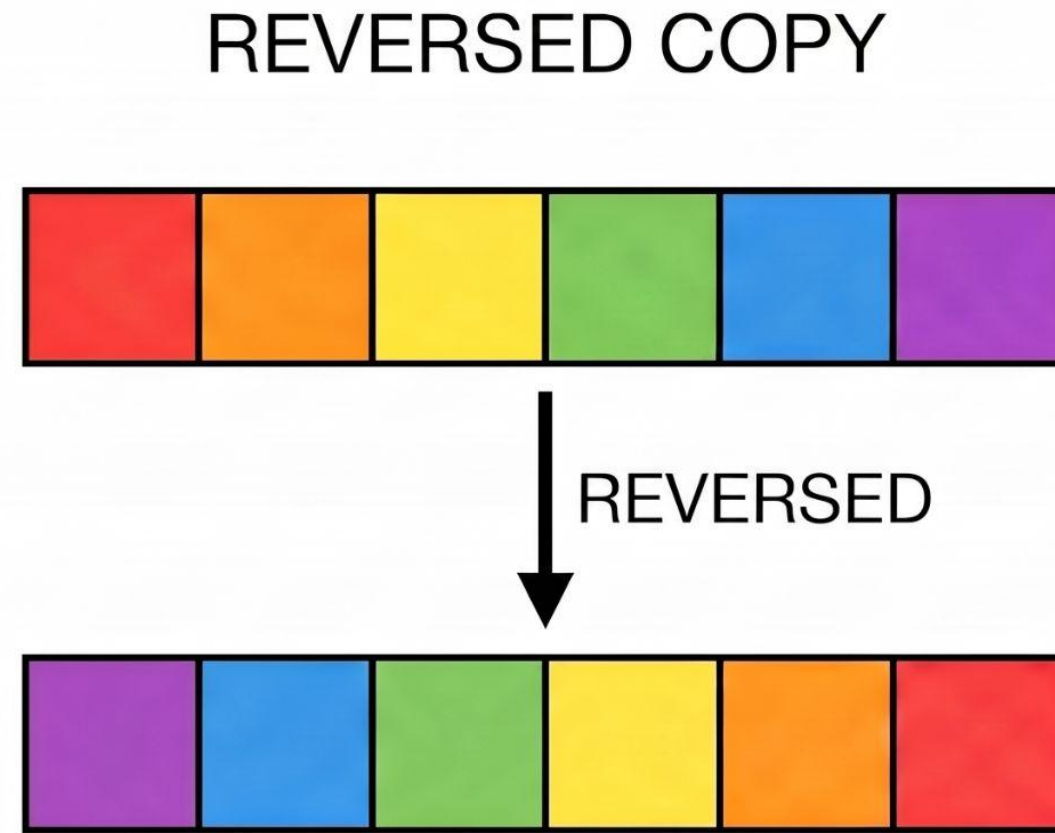
Oracle loss: 2.773 nats (half)

Total Information



Epiplexity (S_t) ≈ 0.5

$S_t + H_t = 1.0$ (total info)



Vocabulary: 256 symbols
 Uniform loss: $\ln\left(\frac{1}{256}\right) \approx 5.545$ nats
 Oracle loss: 2.773 nats (half)

S_t (Epiplexity): ≈ 0.0
 H_t (Entropy): ≈ 1.0 (all of it)

*Epiplexity reveals the
 ARCHITECTURE'S
 naturally revealed*

Because TRUE epiplexity is incomputable and observer-dependent, we need to be careful. We can't just say "this pattern has high epiplexity/structure" — we have to specify the observer and the compute budget. Regardless, these are our tools to speak about the *complexity* of data.

How do we run the experiments?

STEP 1: Pre-Pretraining

>> Train a model (decoder-Llama) on synthetic patterns (Dyck brackets, copies, counting, NCA...)

STEP 2: RESET

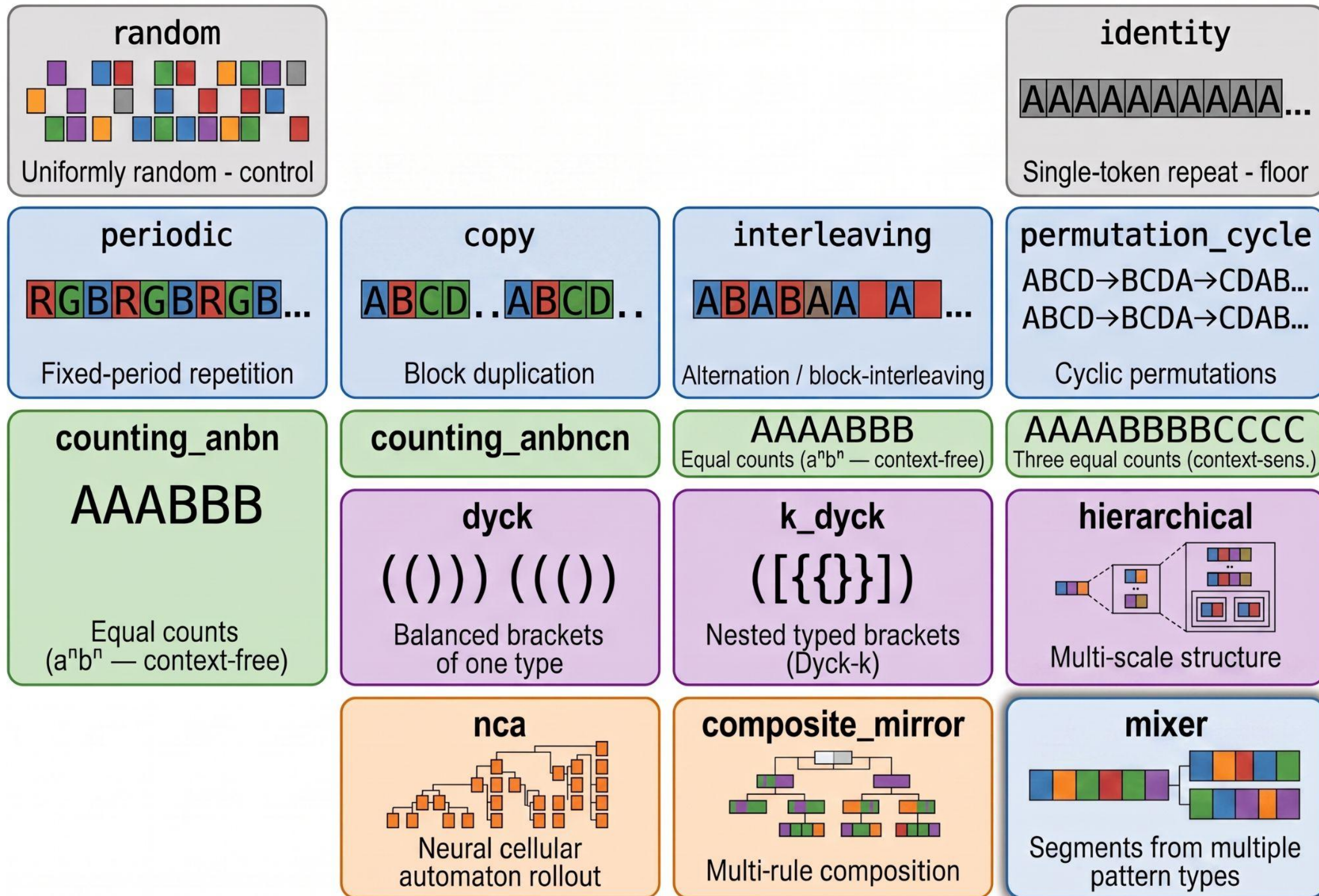
>> Keep attention weights, reinitialize embeddings.

STEP 3: Pretraining

>> Train ALL conditions on a fixed budget of real web text (e.g., c4)

STEP 3: Evals

>> Measure val. loss and downstream performance on canarie benchmarks.



How do we run the experiments?

For ALL these patterns we “know”:

- >> **Gzip complexity** : Raw compressibility; a cheap proxy for total information content.
- >> **Oracle loss** : The theoretical minimum loss if the model knew the exact data-generating rule.
- >> **$S_{\square}$ (epiplexity)**: The bits the model spent learning the pattern; the area under the loss curve.
- >> **$H_{\square}$ (entropy)**: The bits of residual unpredictability after the model extracted all learnable structure.

Can we demonstrate that any of these measures correlate with—or even causally influence—changes in downstream performance?

What did we find?

The experiment was designed to answer a cascade of questions:

- >> Q0: Is there a universal "sweet spot" of undertraining that predicts transfer?
- >> Q1: Do gzip and epiplexity measure the same thing?
- >> Q2: Does the complexity-matching hypothesis hold?
- >> Q3: Can ANY Phase-1 measurement predict downstream transfer?
- >> Q4: Is Phase-1 structural information ($S_{\square}$) conserved into Phase 2?
- >> Q5: WHEN does the advantage appear during Phase 2 training?

What we know so far is that pre-pretraining on synthetic patterns yields small but consistent improvements, although many of the results are counterintuitive and contradict findings reported in parts of the literature ([Find the Full Story](#)).

Thank You

Nicholas Kluge Corrêa
Postdoc @ Bonn-Aachen International Center for Information Technology (b-it)